

Heuristics for the Maximization of Operating Rooms Utilization Using Simulation

Jean-Paul Arnaout

Industrial Engineering Department,
Lebanese American University,
PO Box 36, 101-F Byblos, Lebanon
jparnaout@lau.edu.lb

This paper addresses the problem of maximizing the utilization of operating rooms, which is translated to jobs scheduling in an identical parallel machine environment with sequence-dependent setup times and an objective of minimizing the makespan. The jobs' processing times and setup times are stochastic for better depiction of the real world. This is a non-deterministic polynomial time (NP)-hard problem, and in this paper a new heuristic is developed and compared to existing ones using simulation and optimization. The results and analysis obtained from the computational experiments proved the superiority of the proposed algorithm Longest Expected Processing with Setup Time (LEPST) over the other algorithms presented.

Keywords: simulation, optimization, setup time, healthcare, parallel machines, stochastic times, heuristics

1. Introduction and Background

Now more than ever, the healthcare industry needs to respond to the swell in challenges and pressures in its environment. The constant increase in the number of patients and surgeries requires new, unconventional methods that aim at increasing efficiency and, in particular, improving Operating Room (OR) utilization. The latter is one of the largest revenue and cost centers in the healthcare industry [1], i.e. it has a great potential for reducing expenses, especially in that it affects many parts in the hospital, such as resource management, financial management, purchasing, and many other areas along with the patients' waiting times.

The current norm in hospitals is that patients wait long hours before they are checked in and prepared for surgery. Unsuccessful scheduling of ORs is simultaneously decreasing the rooms' utilization and increasing the patients' delays, an act that is very costly. Operations managers' main target is to attain more accurate scheduling techniques in order to improve the ORs utilization.

Following the above, this paper addresses the problem of ORs' scheduling (non-emergency cases) with the objective of maximizing the utilization of the rooms using a dispatching rule. A dispatching rule is a priority rule, based on which operations are organized in the different rooms. As the salaries of the staff account for most OR costs, it is of great importance that the number of staff remains at its critically needed value, because in the OR scheduling case, the best way to maximize personnel utilization is to maximize the OR utilization [2]. Currently, most hospitals assign block times to surgeon groups based on their requests. Then, surgeons contact the administrative office for a specific starting time for surgery and request a reservation within the allocated block time. The reservation within a certain time block is done on a 'first come first served' basis most of the time.

In our study, the operations that are non-emergency cases are scheduled and planned on a daily basis. The hospital has m ORs where n patients will have operations. Each patient's operation has a certain stochastic duration, and also a preparation phase before the operation. The duration of the preparation is affected by the sequence of the operations in a particular OR. This scheduling problem when translated to machine scheduling becomes the scheduling of n jobs with sequence-dependent setup times on m identical parallel machines with the objective of minimizing the maximum completion time. This is at least a

SIMULATION, Vol. 86, Issue 8-9, August-September 2010 573–583

© 2010 The Society for Modeling and Simulation International

DOI: 10.1177/0037549709352497

Figure 1 appears in color online: <http://sim.sagepub.com>

non-deterministic polynomial time (NP)-hard problem as the simplified problem of two identical machines with no setup times is NP-hard in the ordinary sense [3, 4]. Discrete event simulation will be used to model and test three heuristics for the problem addressed in this paper. Some preliminary promising results for this problem have been reported by [5].

The rest of this paper is organized as follows. In Section 2, the related research is summarized. In Section 3, the problem statement and objective function are presented. Section 4 contains a description of the heuristics developed and used. The computational results and output analysis are described in Section 5. Finally, we conclude our results in Section 6.

2. Literature Review

The maximization of the OR utilization has been the aim of several researchers. Few of them used simulation as a tool to model and solve healthcare scheduling problems. Shmitz and Kwak [6] developed a Monte Carlo simulation model for OR and recovery room usage. The authors used their model to assess the impact of increased bed capacity on the operating and recovery rooms' facilities and space. Dexter et al. [2] used computer simulation for scheduling ORs' block time in order to improve utilization; no preparation time was considered in their paper. Dexter [7] analyzed the effects of scheduling on ORs' labor costs and the effect of having a high OR utilization on the salaries of the personnel. Moreover, the author addressed some operational goals. Tsoy et al. [1] introduced a genetic algorithm approach for the surgery ORs' scheduling problem. The authors assumed the operations times to be deterministic. Jebali et al. [8] introduced a two-step approach for OR scheduling, where operations are assigned to ORs in the first step, and then these operations are sequenced in the second step. The authors considered all times to be deterministic. For more reading on techniques to construct the OR schedule, the reader may refer to [9, 10].

The most common objectives studied in parallel machine scheduling are the minimization of completion time, tardiness, and makespan. The identical parallel machine problem with the objective of minimizing the makespan has been proven to be a NP problem [3, 11]. Due to this complexity, it became a common and acceptable practice to find suitable heuristics instead of optimal solutions for such problems. Following the conventional scheduling notation, the identical parallel machine problem can be denoted by $P_m/C_{\max}$, where m refers to the number of machines, and $C_{\max}$ to the maximum completion time (makespan). In this problem (without setup times) with jobs having a deterministic processing time, the largest processing time (LPT) heuristic gives near optimal results [12, 13]. As for the same problem but with jobs having a stochastic processing time, the longest expected processing time (LEPT) rule minimizes the expected makespan

in the class of non-preemptive jobs [13]. Lee and Pinedo [14] addressed the same problem with setup times, where all times are deterministic, and presented a three-phase heuristic for minimizing the sum of the weighted tardiness. Monma and Potts [15] addressed the problem of two identical parallel machines with batch setup times and claimed that pseudo-polynomial algorithms exist for minimizing the makespan when the number of batches is fixed. Min and Cheng [16] introduced a genetic algorithm for the same objective but without setup times and where the processing times are deterministic. Rabadi et al. [17] solved the unrelated version of the problem with setup times optimal for small-size instances and compared two heuristic approaches for large problems – only the deterministic times were considered.

Stochastic machine scheduling problems have been considered, among others, by [4, 18–24]. However, and up to our knowledge, no previous research has addressed the generation of schedules in identical parallel machines with stochastic processing and sequence-dependent setup times, and this is where the contribution of this paper lies.

3. Problem Statement

The main objective of this research is to find a dispatching rule to schedule the operations in the ORs in a way that maximizes the utilization of the rooms. A flow containing significant processes of the OR preparation, procedure, and discharge is shown in Figure 1. We recall that this paper only tackles the scheduling of patients, as highlighted in Figure 1.

3.1 The OR Scheduling Problem

Scheduling is present in many areas of the hospital, one of which is the OR. The OR scheduling problem is translated to machine scheduling as follows:

- The ORs are regarded as machines.
- The operations are regarded as jobs.
- Operations can be performed in any room, as on any machine.
- Operations can be performed at any time, which is translated as job independence.
- Operations need a preparation of equipment and personnel before they start, which translates to jobs having a sequence-dependent setup time. Depending on the type of operation and its preceding one, the preparation time could either increase or decrease. For example, scheduling a breast augmentation surgery after an open heart surgery will definitely require extra preparation (setup) time between the operations. In contrast, the preparation time would decrease if the open heart surgery was scheduled next, as the latter requires more clean up.

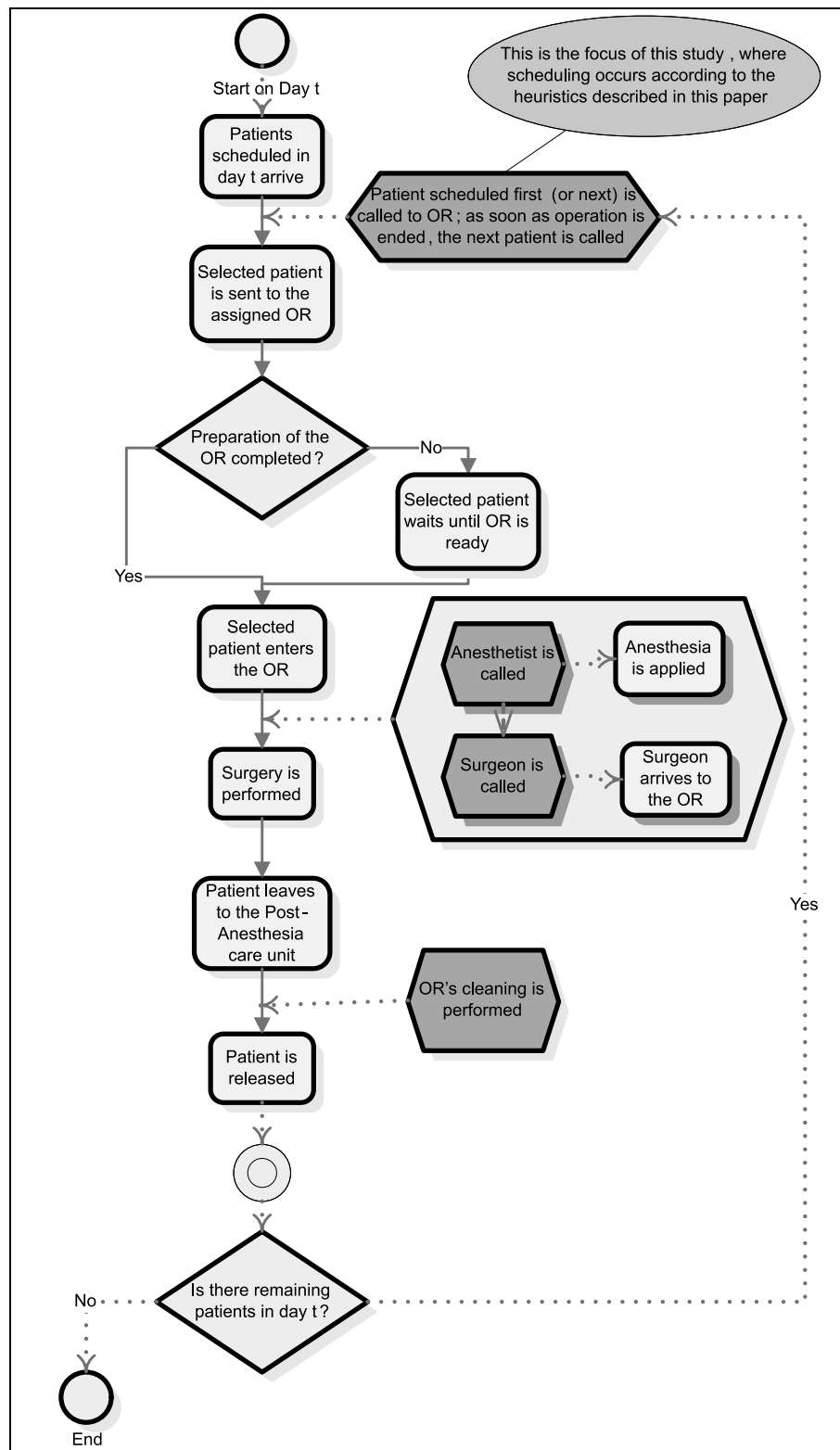


Figure 1. Flow of the OR process

- The objective is to organize operations in such a way as to complete all the operations as soon as possible, by minimizing the maximum completion time at the end of the working day. This can be translated to the minimization of the maximum completion time $C_{\max}$ (or makespan).

Some of the constraints/assumptions for this problem are listed as follows. Firstly, the operations are considered independent from one another; the ORs are identical and parallel, meaning any operation can be scheduled on any OR at any time; the scheduling of operations is a daily process; the quantity of operations scheduled is based on the daily capacity; and all the operations are scheduled before the day begins.

The problem statement in machine-scheduling terms is the following:

$$P_m/S_{ik}/C_{\max}$$

where in P_m the m are parallel identical machines and in S_{ik} the jobs are constrained with sequence-dependent stochastic setup times. The objective is to minimize $C_{\max}$ (maximum completion time or makespan).

3.2 Problem Constraints

There are m identical parallel machines, and n independent jobs to be scheduled on the machines. Each machine is able to process any type of job and a job has the same processing time on any machine (Identical). The jobs are simultaneously available at the beginning of the scheduling horizon (at time zero). Furthermore, each job can be processed on any of the machines, but needs to be processed by one machine only, and each machine is capable of processing only one job at a time. Job preemption is not allowed and there is no processing precedence on any of the machines. The machine setup times are dependent on the jobs' sequence, where setup times depend on both the job just completed and the next job to be processed. Setup times are assumed to be machine independent, such that regardless of which machine jobs k and i are assigned, S_{ki} would be the setup time required if job i is scheduled after job k .

Our main objective is to have the maximum utilization of the ORs. The makespan, defined as $\max\{C_1, C_2, \dots, C_n\}$, where C_j is the completion time for job j , is equivalent to the completion time of the last job to leave the system. A minimum makespan usually implies a high utilization of the machines [13]. Hence, by minimizing $C_{\max}$, we maximize the utilization of the ORs.

4. Heuristic Algorithms

In the following sections, different heuristics are presented and compared in order to determine the most appropriate one for our problem. The jobs' processing and setup times

are stochastic and drawn from different uniform distributions. Whenever a job is called by any algorithm to be sorted with the other jobs or sent to a machine, it will be assigned a processing time and a setup time following some uniform distribution; this is discussed more in Section 5.

4.1 Longest Expected Processing Time and Shortest Expected Processing Time

The Longest Processing Time First (LPT) rule gives the priority to the job that has the longest processing time to be scheduled first. Under LPT, the shortest job is the last job to be assigned to the machines.

Shortest Processing Time First (SPT) gives the priority to the job that has the shortest processing time to be scheduled first. Under SPT, the longest job is the last job to be assigned to the machines.

During the schedule's execution for the multiple machine case, LPT and SPT assign the jobs to the first available machine.

LEPT and shortest expected processing time (SEPT) are very similar to LPT and SPT, but they differ in that they consider the average processing times of jobs. In other words, the advantages over the LPT and SPT are the ability to schedule jobs in stochastic environments [13].

In order to use LEPT or SEPT, we must calculate the averages of the expressions of processing times. In particular, whenever we want to schedule a job on the machines, LEPT will pick from the unscheduled jobs the one with the maximum EP_i (where EP_i is the expected processing time of job i on any machine for $i = 1, \dots, n$), while SEPT will pick the job with the minimum EP_i ; the chosen job using either rule will be sent to the machine with the smallest completion time.

The limitation of LEPT and SEPT for the OR problem is that they do not account for setup (preparation) times.

4.2 Longest Expected Processing with Setup Time

The Longest Expected Processing with Setup Time (LEPST) is the dispatching rule that is introduced for OR scheduling in this study; it is an extension of LEPT where the setup times are considered. The main challenge of the problem is the consideration of the sequence-dependent setup time. It is shown that the $P_m/S_{j,k}/C_{\max}$ problem is strongly NP-hard [13].

Let (S) be a set containing the unscheduled jobs:

1. Find job $i \in (S)$ where (1) is at its maximum:

$$(EP_i \times \alpha) + (1 - \alpha) \times S_{M_i} \quad (1)$$

where S_{M_i} is the maximum setup time of job i when scheduled on any of the machines (as its setup depends on the last job of each machine), i.e. $S_{M_i} =$

Max $\{s_{K_j,i}\}$, and k_j is the last job on machine j , and the control parameter α value was determined using the Design of Experiments (DoE) (Section 4.2.1.) and optimization (Section 4.2.2.).

2. Remove i from (S) and assign it to machine j where (2) is at its minimum:

$$C_{K_j} + s_{K_j,i} \quad (2)$$

where C_{K_j} refers to the completion time of the last job K_j

3. If $S = \emptyset$, STOP; else go to Step 1.

The rationale behind LEPST can be described as follows. Using (1), the job with the largest weighted (α) processing time and weighted $(1 - \alpha)$ setup time is removed from (S) (without deciding yet on the machine assignment), i.e., LEPST removes first the job that has a large processing time and that might have a large setup time (as the latter depends on the machine assignment). Next, the removed job is assigned to the machine where (2) is at its minimum; i.e., the job will most likely be assigned to a machine where its setup is minimized. The intention is to dispose early on of a job that might produce a large setup if left until the end of scheduling, where the machines already have many assigned jobs; i.e. the job might end up on the machine where the maximum setup (S_{M_i}) occurs. To summarize, by giving the priority to jobs that have the maximum setup time, there will be fewer jobs having high setup times assigned at the end of the schedule, hence, affecting less the $C_{\max}$.

4.2.1 LEPST Design of Experiments

The DoE was utilized to determine the appropriate values for α parameter in LEPST that will minimize the makespan $C_{\max}$. Numerous publications provide a good review of DoE (e.g., [25–27]).

The factors considered in this experiment along with their levels of low, medium, and high, respectively, are as follows: *Times*: (*Processing*, *Balanced*, *Setup*), α : (0, 0.5, 1), *Rooms* : (2, 4, 8), and

Patients : ($U[8, 12]$, $U[60, 80]$, $U[160, 240]$).

The values of the parameter levels were selected based on many runs under different settings. Note that an ($\alpha = 0$) in Equation (1) indicates that only *Setup Times* are considered when selecting jobs (operations) for scheduling, and ($\alpha = 1$) ignores the *Setup Times* and only considers the *Expected Processing Times*. The *Times* factor indicates if the *Processing Times* are dominant with respect to *Setup Times*, if both the processing times and setup times are balanced, or if the *Setup Times* are dominant. These times generation is discussed in Section 5. *Rooms* refer to having 2, 4, or 8 ORs. *Patients* refer to the density of arrival

of patients per day; the latter could be as low as $U[8, 12]$ and as high as $U[160, 240]$. To reduce the number of runs but reach sound conclusions, I-Optimal Design was utilized, which has been shown to be an effective design [27]. JMP 6.0 from SAS was used to generate an I-Optimal design, with 27 experiments. The factors, along with their interactions, were analyzed using regression, analysis of variance (ANOVA), and factors' effect tests. Three-factor interactions and higher were not considered as they typically have weak effects [28]. The tests indicated that the overall model fit and factor selection is meaningful with a high R^2 value (0.9869) and low p -value (0.0023) for the overall model.

Based on a 95% Confidence Interval (CI), a relatively large t -Stat, and a small p -value (less than 0.05), the parameters were estimated and a prediction expression was generated and solved for the minimum makespan while varying the factor α values. As a result, α parameter values were determined to provide the best performance for LEPST as shown in Table 1.

4.2.2 LEPST Optimization Approach

While the DoE gives great insight into the appropriate values of α for each problem structure, we could reach better α values by running an optimization for each of the 12 problem structures that will be tested in this study. *OptQuest* from OpTek Systems, Inc. is a computer software system that allows users to automatically search for optimal solutions to complex systems, and is integrated within Arena, the simulation software that was used in this study. *OptQuest* combines the metaheuristic procedures of Tabu Search, Neural Networks, and Scatter Search into a single composite method, and has demonstrated the power of this method in an extensive range of practical applications. Figure 2 depicts the link between the optimization and simulation software.

For every problem structure, the optimization software based on its integrated heuristics decides on a value of α . The latter is used in the simulation run and the makespan is recorded. The makespan is inputted back into the optimization software to decide on an appropriate value of α . This loop is repeated for as many replications as needed in order to guarantee a minimum makespan with a 95% CI that is at most 5% from the average makespan. The mathematical formulation is as follows:

$$\text{Min Makespan} \quad (3)$$

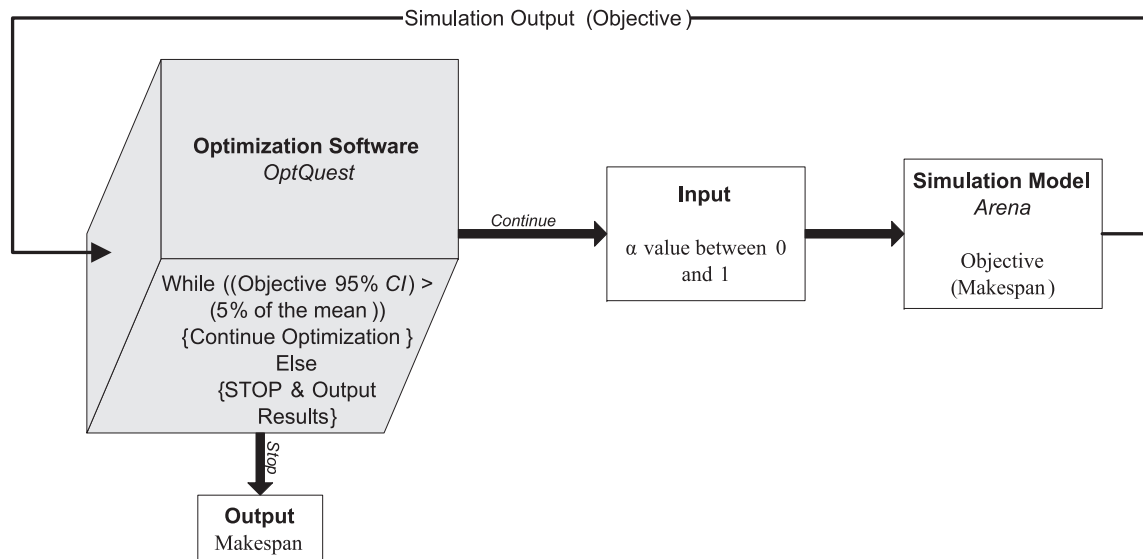
Subject to:

$$0 \leq \alpha \leq 1. \quad (4)$$

The α values generated using this approach are presented in Table 2. These results were similar, but obviously more advantageous than the ones reached using the DoE. Following this, the Table 2 results are used in the computational tests when comparing the heuristics.

Table 1. α values for LEPST generated using DoE (P: processing dominant, B: balanced, S: setup dominant)

Times	Rooms	Patients	α	Times	Rooms	Patients	α
P	2	$U[8,12]$	0.5	P	8	$U[60,80]$	0.5
B	2	$U[8,12]$	1	B	8	$U[60,80]$	0.3
S	2	$U[8,12]$	1	S	8	$U[60,80]$	0.5
P	4	$U[8,12]$	0.3	P	2	$U[160,240]$	1
B	4	$U[8,12]$	1	B	2	$U[160,240]$	1
S	4	$U[8,12]$	0.7	S	2	$U[160,240]$	1
P	8	$U[8,12]$	1	P	4	$U[160,240]$	1
B	8	$U[8,12]$	0.5	B	4	$U[160,240]$	1
S	8	$U[8,12]$	0.5	S	4	$U[160,240]$	1
P	2	$U[60,80]$	0.3	P	8	$U[160,240]$	0.5
B	2	$U[60,80]$	0.5	B	8	$U[160,240]$	1
S	2	$U[60,80]$	1	S	8	$U[160,240]$	1
P	4	$U[60,80]$	1				
B	4	$U[60,80]$	0.3				
S	4	$U[60,80]$	1				

**Figure 2.** Interaction between the simulation and optimization software

5. Computational Tests

The proposed rules were implemented in Arena 10.0 Simulation Software from Rockwell Systems, running on Windows XP with a Pentium 4 processor. The popularity of simulation has been increasing over the past decade, mainly due to its ability to deal with very complicated models of correspondingly complicated systems [29]. The reason stochastic data was used for the processing and setup times is to ensure a more real representation of the healthcare environment, where most of the time an operation will not finish on a specific time, but on a range be-

tween two times. Simulation is considered to be one of the best approaches to deal with such a source of randomness. The jobs' (operations) processing times and machines' setup (preparation) times are stochastic following uniform distributions; the dominant times (depending if the times are processing or setup dominant) can take any value of four different uniform distributions (hrs): $U[8, 10]$, $U[6, 8]$, $U[5, 7]$, and $U[5, 15]$, the minor (non-dominant) times can take any value of the following distributions: $U[1, 2]$, $U[0.5, 3]$, $U[1, 3]$, and $U[4, 6]$, and in the case of balanced times, both processing and setup times can take any value of the following distributions: $U[1, 2]$, $U[0.5,$

Table 2. α values for LEPST generated using optimization

Rooms	Patients	α		
		Processing dominant	Setup dominant	Balanced
2	$U[8,12]$	0.32	1	1
	$U[40,60]$	0.25	1	0.38
	$U[80,120]$	1	1	1
	$U[160,240]$	1	1	1
4	$U[8,12]$	0.34	0.77	0.88
	$U[40,60]$	0.27	0.75	0.127
	$U[80,120]$	1	1	0.35
	$U[160,240]$	1	1	1
8	$U[8,12]$	1	0.4	0.44
	$U[40,60]$	0.2	0.41	0.36
	$U[80,120]$	0.49	0.43	0.37
	$U[160,240]$	0.56	1	1

3], $U[1, 3]$, and $U[4, 6]$. Recall that these values are not known until the job (operation) is actually started on the machine (OR); this is how the algorithms' robustness is being tested. The reason uniform distributions were used is due to their high variances, ensuring that the presented heuristics are being tested under unfavorable conditions [30].

Our simulation model logic is described in Figure 3. The three heuristics were tested under 2, 4, and 8 parallel machines, as well as different job combinations. The number of replications for each of the problem structures was equal to 30 replications. The number of replications was obtained using the approach presented by [29] in order to obtain good CIs, where the simulation was initially run for 10 replications. The half width obtained was fairly large, and therefore more runs were needed to reach a tolerable half width (i.e. a half width that is at most 5% of the average). Equation (5) was used to determine the number of runs:

$$k \cong k_0 \times \frac{h_0^2}{h^2} \quad (5)$$

where k_0 and h_0 refer respectively to the initial replication number (10) and its associated half width, h refers to the desired (5% from average) half width, and k is the number of needed replications ($k = 30$).

The results obtained from running 30 replications at 95% confidence level are shown in Table 3. The latter clearly indicates that LEPST significantly outperformed the other algorithms for the majority of experiments. LEPT was second best and it outperformed SEPT.

Figures 4–6 depict the relative performance among the rules. The relative performance was calculated as follows:

$$\text{Relative Performance} = \frac{Z_l}{Z_{\min}},$$

for $l = 1, 2$, and 3 ,

where Z_l refers to the makespan obtained when using heuristic l , and $Z_{\min}$ refers to the minimum makespan among all three heuristics.

Figures 4–6 confirm that LEPST had the best performance, with LEPT second. Furthermore, the larger the number of rooms (machines), the better LEPST performs when compared to the other rules. This makes sense as the setup times have a higher impact when the number of machines increases. In addition, and when compared to LEPT, LEPST had the best performance with dominant processing times, then balanced and finally setup. This unexpected conclusion is due to the fact that in dominant processing times, while LEPT completely disregarded the setup times, LEPST still accounts for them.

As there were some cases where LEPT and SEPT performed as well as LEPST, paired t -tests over the 12 problem structures were conducted with 95% CI to determine the finest rule. These tests are presented in Table 4. The results indicate that the mean difference is statistically significant for all tests, as none of the 95% CIs overlap with zero. Furthermore, the results reconfirm the superiority of LEPST, followed by LEPT, and lastly SEPT.

6. Conclusions

In this paper, we have introduced an effective heuristic algorithm, LEPST, for maximizing the utilization of ORs, which was shown to be equivalent to minimizing the makespan on identical parallel machines with sequence-dependent setup times. LEPST was compared to two other algorithms, LEPT and SEPT. All three algorithms were modeled and tested through simulation, and our conclusions were drawn using a large number of replications and several statistical tests. These tests revealed that the algorithms are extremely fast as they found solutions for 240 jobs and eight machines in a small fraction of a second. Computational experiments also showed that LEPST significantly outperformed the other algorithms, especially as the number of machines increased.

It is worth noting here that this study was generated as part of a simulation project conducted at a local hospital in Lebanon with four identical ORs, and the rules were compared using real data drawn from this hospital. The conclusions reached were similar to the ones in this paper, in the sense that LEPST significantly outperformed the other algorithms, and LEPT was the second best.

Furthermore, the rules were also tested in a deterministic environment, and the conclusions remained unchanged.

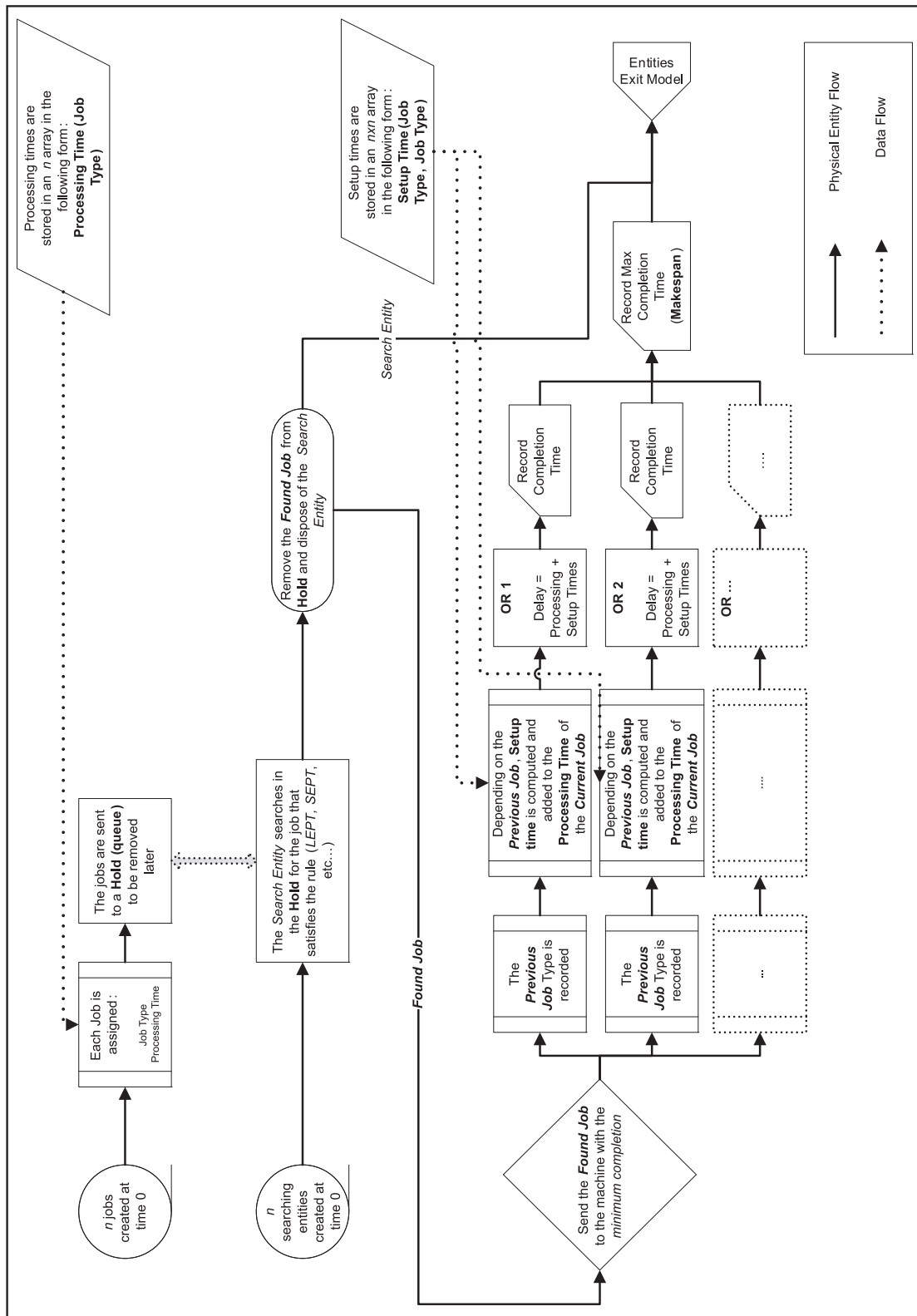


Figure 3. Simulation model logic

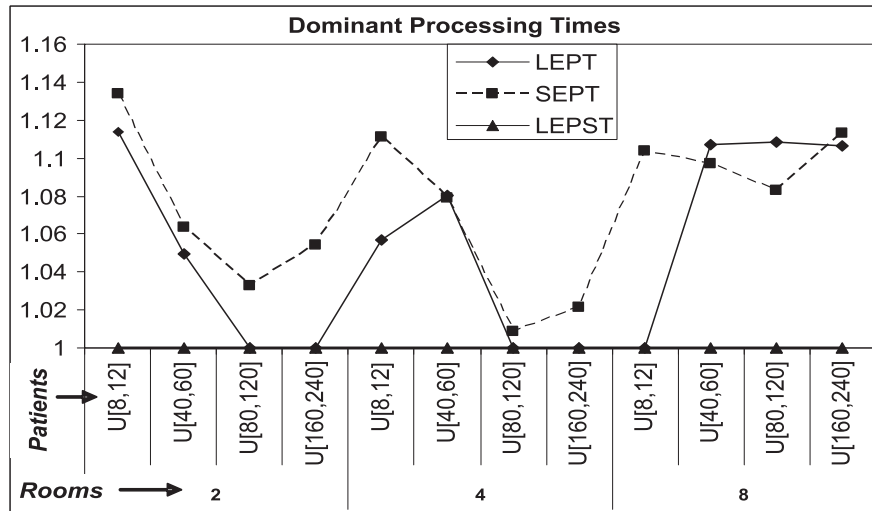


Figure 4. Relative performance for dominant processing times

Table 3. $C_{\max}$ Averages from computational experiments

Rooms	Patients	LEPT			SEPT			LEPST		
		Processing dominant	Setup dominant	Balanced	Processing dominant	Setup dominant	Balanced	Processing dominant	Setup dominant	Balanced
2	U[8,12]	54.1317	24.16	22.986	55.1152	28.9827	23.893	48.6	24.16	22.36
	U[40,60]	255.17	142.67	126.88	258.49	148.4	128.08	243.1	142.67	118.15
	U[80,120]	490.37	271.72	241	506.42	289.34	255.99	490.37	271.72	241
	U[160,240]	958	551.2	496.67	1009.97	573.88	507.41	958	551.2	496.67
4	U[8,12]	30.3475	14.9948	13.069	31.9093	16.6795	14.082	28.71	13.52	12.49
	U[40,60]	135.01	76.9313	65.069	134.86	76.7914	66.276	125	71.5	58.41
	U[80,120]	261.15	145.49	129.76	263.47	147.61	130.73	261.15	145.49	124.39
	U[160,240]	505.44	289.87	255.56	516.32	290.44	257.74	505.44	289.87	255.56
8	U[8,12]	19.2061	8.9678	8.4437	21.2024	10.5278	9.7212	19.2061	8.27	8.05
	U[40,60]	73.0591	39.7305	34.487	72.3806	40.7443	34.56	65.98	36.47	30.36
	U[80,120]	138.44	75.714	66.51	135.23	76.9309	67.704	124.89	71.02	62.03
	U[160,240]	263.32	147.29	131.08	264.91	149.16	131.76	237.97	147.29	131.08

Table 4. Paired t -tests with 95% CI

Paired t -tests	Mean difference (MD)	Standard deviation	t -Stat	Two-tailed p	95% CI on MD
LEPT – LEPST (Processing dom.)	5.19084	4.98936	3.60400	0.00414	[2.02,8.36]
LEPT – LEPST (Setup dominant)	3.54099	4.48267	2.73640	0.01936	[0.693,6.389]
LEPT – LEPST (Balanced)	4.68707	4.56972	3.55306	0.00453	[1.784,7.59]
SEPT – LEPST (Processing dom.)	7.51682	3.96088	6.57405	0.00004	[5,10.03]
SEPT – LEPST (Setup dominant)	9.63451	9.12700	3.65672	0.00378	[3.83,15.43]
SEPT – LEPST (Balanced)	8.33912	6.05598	4.77008	0.00058	[4.491,12.187]

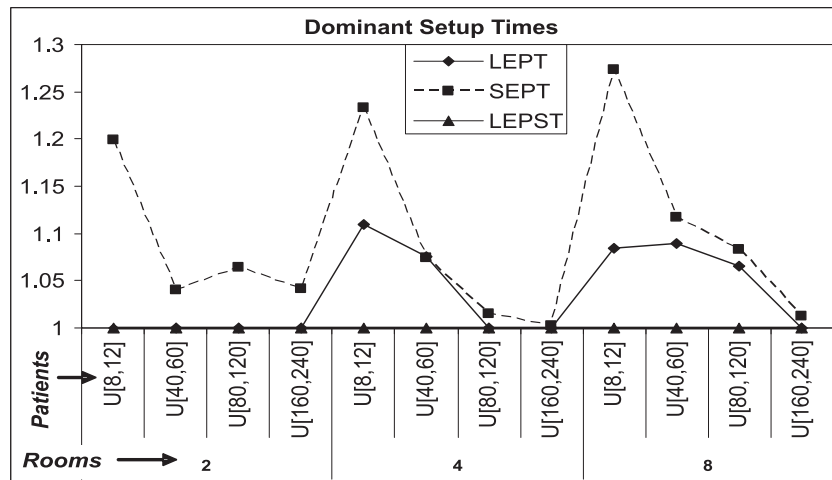


Figure 5. Relative performance for dominant setup times

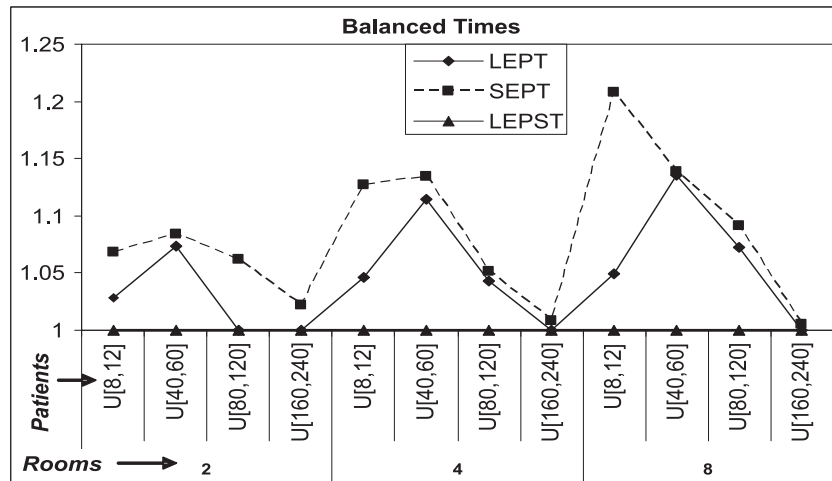


Figure 6. Relative performance for balanced times

7. References

- [1] Tsoy, G., J.-P. Arnaout, T. Smith and G. Rabadi. 2004. A genetic algorithm approach for surgery operating rooms scheduling problem. In *Proceedings of the 25th National Conference of the American Society for Engineering Management*, Alexandria, Virginia.
- [2] Dexter, F., A. Macario, R. Traub, M. Hopwood and D. Lubarsky. 1999. An operating room scheduling strategy to maximize the use of operating room block time: computer simulation of patient scheduling and survey of patients' preferences for surgical waiting time. *Anesthesia & Analgesia*, 89: 7–20.
- [3] Garey, M.R. and D.S. Johnson. 1979. *Computer and intractability: a guide to the theory of NP-completeness*, W H Freeman, San Francisco.
- [4] Bruno, J.L., P.J. Downey and G.N. Frederickson. 1981. Sequencing tasks with exponential service times to minimize the expected flow time or makespan. *Journal of the ACM*, 28: 100–113.
- [5] Arnaout, J.-P. and Kulbashian, S. 2008. Maximizing the utilization of operating rooms with stochastic times using simulation. *Proceedings of the 2008 Winter Simulation Conference*, pp. 1617–1623, Miami, FL.
- [6] Shmitz, H. and N. Kwak. 1972. Monte Carlo simulation of operating-room and recovery-room usage. *Operations Research*, 20: 1171–1180.
- [7] Dexter, F. 2000. A strategy to decide whether to move the last case of the day in an operating room to another empty operating room to decrease overtime labor costs. *Anesthesia & Analgesia*, 91: 925–928.
- [8] Jebali, A., A. Alouane and P. Ladet. 2006. Operating rooms scheduling. *International Journal of Production Economics*, 99: 52–62.
- [9] Guinet, A. and S. Chaabane. 2003. Operating theatre planning. *International Journal of Production Economics*, 85: 69–81.
- [10] Kharraja, S., S. Chaabane and E. Marcon. 2002. Evaluation de performances pour deux stratégies de programmation opératoire de bloc. In *Actes de la 2^{ème} Conf Int Francophone d'Automatique*, Nantes, France.
- [11] Sethi, R. 1977. On the complexity of mean flow time scheduling. *Mathematics of Operations Research*, 2: 320–330.

- [12] Graham, R.L. 1969. Boundaries on multiprocessing timing anomalies. *SIAM Journal of Applied Mathematics*, 17: 416–429.
- [13] Pinedo, M. 1995. *Scheduling: Theory, Algorithms, and Systems*, Prentice Hall, NJ.
- [14] Lee, Y. and M. Pinedo. 1997. Scheduling jobs on parallel machines with sequence-dependent setup times. *European Journal of Operational Research*, 100: 464–474.
- [15] Monma, C.L. and C.N. Potts. 1989. On the complexity of scheduling with batch setups. *Operations Research*, 37: 798–804.
- [16] Min, L. and W. Cheng. 1999. A genetic algorithm for minimizing the makespan in the case of scheduling identical parallel machines. *Artificial Intelligence in Engineering*, 13: 399–403.
- [17] Rabadi, G., Moraga, R. and Al-Salem, A. (2006), Heuristics for the unrelated parallel machine scheduling problem with setup times. *Journal of Intelligent Manufacturing*, 17: 85–97.
- [18] Glazebrook, K.D. 1979. Scheduling tasks with exponential service times on parallel machines. *Journal of Applied Probability*, 16: 685–689.
- [19] Weiss, G. and M. Pinedo. 1980. Scheduling tasks with exponential service times on non-identical processors to minimize various cost functions. *Journal of Applied Probability*, 17: 187–202.
- [20] Weber, R.R., P. Varaiya and J. Walrand. 1986. Scheduling jobs with stochastically ordered processing times on parallel machines to minimize expected flowtime. *Journal of Applied Probability*, 23: 841–847.
- [21] Weiss, G. 1992. Turnpike optimality of Smith's rule in parallel machines stochastic scheduling. *Mathematics of Operations Research*, 17: 255–270.
- [22] Mohring, R., A. Shulz and M. Uetz. 1999. Approximation in stochastic scheduling: the power of LP-based priority policies. *Journal of the ACM (JACM)*, 46: 924–942.
- [23] Javidi, T., N-O. Song and D. Teneketzis. 2001. Expected makespan minimization on identical machines in two interconnected queues. *Probability in the Engineering and Informational Sciences*, 15: 409–443.
- [24] Arnaout, J-P., G. Rabadi and J.H. Mun. 2006. A dynamic heuristic for the stochastic unrelated parallel machine scheduling problem. *International Journal of Operations Research*, 3: 136–143.
- [25] Fisher, R.A. 1960. *The Design of Experiments*, Hafner Pub. Co., NY.
- [26] Taguchi, G. 1993. *Taguchi Methods: Design of Experiments*, American Supplier Institute, Inc., MI.
- [27] NIST/SEMATECH e-Handbook of Statistical Methods, <http://www.itl.nist.gov/div898/handbook/>.
- [28] Ross, P. 1996. *Taguchi techniques for quality engineering*, McGraw Hill, New York.
- [29] Kelton, D., R. Sadowski and D. Sturrock. 2004. *Simulation with Arena*, 3rd edn, McGraw-Hill Companies, New York.
- [30] Weng, M., J. Lu and H. Ren. 2001. Unrelated parallel machine scheduling with setup consideration and a total weighted completion time objective. *International Journal of Production Economics*, 70: 215–226.

J-P Arnaout is an Assistant Professor at the Industrial and Mechanical Engineering Department at the Lebanese American University, Byblos, Lebanon. He received his PhD and M.S. from the Department of Engineering Management and Systems engineering at Old Dominion University, Norfolk, Virginia in 2006 and 2003 respectively. His Research interests include Optimization Techniques, Modeling and Simulation, and Scheduling and Rescheduling.